Assignment -11.2

Hall ticket no:2303A510A6

Batch no:02

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

class Stack:

pass

Expected Output:

- **A functional stack implementation with all required methods and docstrings**

Code:

```python
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        """Add an item to the stack"""
        self.items.append(item)

    def pop(self):
        """Remove and return the top item"""
        if not self.is_empty():
            return self.items.pop()
        return None

    def peek(self):
        """View the top item without removing it"""
        if not self.is_empty():
            return self.items[-1]
        return None

    def is_empty(self):
        """Check if stack is empty"""
        return len(self.items) == 0

    def size(self):
        """Return the number of items in the stack"""
        return len(self.items)

    def display(self):
        """Display all items in the stack"""
        return self.items


# Demonstrate stack operations
if __name__ == "__main__":
    s = Stack()

    # Push operations
    s.push(10)
    s.push(20)
    s.push(30)
    s.push(40)
    print("Stack after pushing 10, 20, 30, 40:", s.display())
```

Output:

```
Stack after pushing 10, 20, 30, 40: [10, 20, 30, 40]
Top element (peek): 40
Stack size: 4
Popped element: 40
Stack after pop: [10, 20, 30]
Is stack empty? False
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/shyam/ai a
Enqueuing: 10, 20, 30, 40, 50
Queue: [10, 20, 30, 40, 50]
Queue size: 5
Peek (front element): 10

Dequeued: 10
Queue: [20, 30, 40, 50]
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

- **FIFO-based queue class with enqueue, dequeue, peek, and size**

Methods

Code:

> Users > shyam > ai assistes code > ass13.py > Queue > __str__

```python
class Queue:
    """FIFO-based queue class with enqueue, dequeue, peek, and size operations."
    def __init__(self):
        self.items = []
    def enqueue(self, item):
        """Add an item to the rear of the queue."""
        self.items.append(item)
    def dequeue(self):
        """Remove and return the front item from the queue."""
        if not self.is_empty():
            return self.items.pop(0)
        return None
    def peek(self):
        """Return the front item without removing it."""
        if not self.is_empty():
            return self.items[0]
        return None
    def size(self):
        """Return the number of items in the queue."""
        return len(self.items)
    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.items) == 0
    def __str__(self):
        """Return string representation of the queue."""
        return str(self.items)
# Demonstration
if __name__ == "__main__":
    q = Queue()
    # Enqueue elements
    print("Enqueuing: 10, 20, 30, 40, 50")
    q.enqueue(10)
    q.enqueue(20)
    q.enqueue(30)
    q.enqueue(40)
    q.enqueue(50)
    print(f"Queue: {q}")
    print(f"Queue size: {q.size()}")
    # Peek at front element
    print(f"Peek (front element): {q.peek()}")
    # Dequeue elements
    print(f"\nDequeued: {q.dequeue()}")
    print(f"Queue: {q}")
    print(f"Queue size: {q.size()}")

    print(f"Dequeued: {q.dequeue()}")
    print(f"Queue: {q}")
    print(f"Queue size: {q.size()}")

    # Peek again
    print(f"Peek (front element): {q.peek()}")
    # Dequeue remaining
    print(f"\nDequeuing all remaining elements:")
    while not q.is_empty():
        print(f"Dequeued: {q.dequeue()}, Queue: {q}")
```

Output:

```
Queue size: 4
Peek (front element): 10

Dequeued: 10
Queue: [20, 30, 40, 50]
Queue size: 4
Dequeued: 10
Queue: [20, 30, 40, 50]
Queue size: 4
Queue: [20, 30, 40, 50]
Queue size: 4
Dequeued: 20
Queue: [30, 40, 50]
Queue size: 3
Peek (front element): 30

Dequeuing all remaining elements:
Dequeued: 30, Queue: [40, 50]
Dequeued: 40, Queue: [50]
Dequeued: 50, Queue: []

Queue after all dequeues: []
Queue: [30, 40, 50]
Queue size: 3
Peek (front element): 30

Dequeuing all remaining elements:
Dequeued: 30, Queue: [40, 50]
Dequeued: 40, Queue: [50]
Dequeued: 50, Queue: []

Queue after all dequeues: []
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node: pass

class LinkedList: pass

Expected Output:

• A working linked list implementation with clear method documentation.

Code:

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node

    def display(self):
        elements = []
        current = self.head
        while current:
            elements.append(str(current.data))
            current = current.next
        print(" -> ".join(elements) if elements else "Empty List")

# Example usage
if __name__ == "__main__":
    ll = SinglyLinkedList()
    ll.insert(10)
    ll.insert(20)
    ll.insert(30)
    ll.display()
```

Output:

```
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/shyam/ai assistes code/ass13.py"
10 -> 20 -> 30
PS D:\Quick Heal\Microsoft VS Code>
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

Expected Output:

- **BST implementation with recursive insert and traversal methods.**

Code:

C: > Users > shyam > ai assistes code > ass13.py > ...

```python
class BST:
    def __init__(self):
        """Initialize an empty Binary Search Tree"""
        self.root = None

    def insert(self, value):
        """Insert a value into the BST"""
        if self.root is None:
            self.root = Node(value)
        else:
            self._insert_recursive(self.root, value)

    def _insert_recursive(self, node, value):
        """Helper method to insert recursively"""
        if value < node.value:
            if node.left is None:
                node.left = Node(value)
            else:
                self._insert_recursive(node.left, value)
        else:
            if node.right is None:
                node.right = Node(value)
            else:
                self._insert_recursive(node.right, value)

    def inorder_traversal(self):
        """Perform in-order traversal and return a list of values"""
        result = []
        self._inorder_recursive(self.root, result)
        return result

    def _inorder_recursive(self, node, result):
        """Recursive helper for in-order traversal"""
        if node is not None:
            self._inorder_recursive(node.left, result)
            result.append(node.value)
            self._inorder_recursive(node.right, result)
# Example usage
if __name__ == "__main__":
    bst = BST()
    values = [50, 30, 70, 20, 40, 60, 80]
    for val in values:
        bst.insert(val)

    print("Inorder Traversal:", bst.inorder_traversal())
```

Output:

```
Inorder Traversal: [20, 30, 40, 50, 60, 70, 80]
PS D:\Quick Heal\Microsoft VS Code>
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
pass
```

Expected Output:

- **Collision handling using chaining, with well-commented methods.**

Code:

```python
class HashTable:
    """Hash table implementation with chaining for collision hanc

    def __init__(self, size=10):
        """Initialize hash table with given size"""
        self.size = size
        # Create list of empty lists (chains) for collision handi
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Hash function to compute index for a given key"""
        return hash(key) % self.size

    def insert(self, key, value):
        """
        Insert a key-value pair into the hash table
        If key exists, update its value
        Uses chaining to handle collisions
        """
        index = self._hash(key)
        chain = self.table[index]

        # Check if key already exists and update it
        for i, (k, v) in enumerate(chain):
            if k == key:
                chain[i] = (key, value)
                return

        # Key doesn't exist, add new pair to the chain
        chain.append((key, value))

    def search(self, key):
        """
        Search for a key in the hash table
        Returns the value if found, None otherwise
```

```python
    def delete(self, key):
        """
        Delete a key-value pair from the hash table
        Returns True if deleted, False if key not found
        """
        index = self._hash(key)
        chain = self.table[index]
        # Search and remove from the chain
        for i, (k, v) in enumerate(chain):
            if k == key:
                chain.pop(i)
                return True
        return False
    def display(self):
        """Display all key-value pairs in the hash table"""
        for i, chain in enumerate(self.table):
            print(f"Index {i}: {chain}")
# Example usage
if __name__ == "__main__":
    ht = HashTable(5)
    # Insert operations
    ht.insert("name", "Alice")
    ht.insert("age", 25)
    ht.insert("city", "New York")
    ht.insert("job", "Engineer")
    print("Hash Table after insertions:")
    ht.display()
    # Search operations
    print(f"\nSearch 'name': {ht.search('name')}")
    print(f"Search 'age': {ht.search('age')}")
    print(f"Search 'phone': {ht.search('phone')}")

    # Delete operation
    print(f"\nDelete 'age': {ht.delete('age')}")
    print("Hash Table after deletion:")
    ht.display()

    # Update operation
    ht.insert("name", "Bob")
    print("\nHash Table after updating 'name':")
    ht.display()
```

Output:

```
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs
Hash Table after insertions:
Index 0: []
Index 1: [('age', 25), ('city', 'New York'), ('job', 'Engineer')]
Index 2: []
Index 3: [('name', 'Alice')]
Index 4: []

Search 'name': Alice
Search 'age': 25
Search 'phone': None

Delete 'age': True
Hash Table after deletion:
Index 0: []
Index 1: [('city', 'New York'), ('job', 'Engineer')]
Index 2: []
Index 3: [('name', 'Alice')]
Index 4: []

Hash Table after updating 'name':
Index 0: []
Index 1: [('city', 'New York'), ('job', 'Engineer')]
Index 2: []
Index 3: [('name', 'Bob')]
Index 4: []
PS D:\Quick Heal\Microsoft VS Code>
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

• Graph with methods to add vertices, add edges, and display connections.

Code:

Users > shyam > ai assistes code > ass13.py > Graph

```python
class Graph:
    def __init__(self):
        self.graph = {}

    def add_vertex(self, vertex):
        """Add a vertex to the graph"""
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, u, v):
        """Add an edge between vertices u and v"""
        if u not in self.graph:
            self.add_vertex(u)
        if v not in self.graph:
            self.add_vertex(v)

        if v not in self.graph[u]:
            self.graph[u].append(v)

    def display(self):
        """Display the graph connections"""
        for vertex in self.graph:
            print(f"{vertex} -> {self.graph[vertex]}")


# Example usage
if __name__ == "__main__":
    g = Graph()

    g.add_vertex('A')
    g.add_vertex('B')
    g.add_vertex('C')
    g.add_vertex('D')

    g.add_edge('A', 'B')
    g.add_edge('A', 'C')
    g.add_edge('B', 'D')
    g.add_edge('C', 'D')

    print("Graph Adjacency List:")
    g.display()
```

Output:

```
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\python.exe "c:/U
Graph Adjacency List:
A -> ['B', 'C']
B -> ['D']
C -> ['D']
D -> []
PS D:\Quick Heal\Microsoft VS Code>
```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

class PriorityQueue:

pass

Expected Output:

- **Implementation with enqueue (priority), dequeue (highest priority), and display methods.**

Code:

```python
#generate code a a priority queue using Python's heapq module Implementation with enqueue (priority), dequeue (highest priority), and display methods.
import heapq
class PriorityQueue:
    def __init__(self):
        self.heap = []
    def enqueue(self, data, priority):
        """Add an element with a given priority to the queue."""
        heapq.heappush(self.heap, (priority, data))
    def dequeue(self):
        """Remove and return the element with the highest priority (lowest value)."""
        if not self.heap:
            print("Priority Queue is empty!")
            return None
        return heapq.heappop(self.heap)
    def display(self):
        """Display all elements in the priority queue."""
        if not self.heap:
            print("Priority Queue is empty!")
            return
        print("Priority Queue (priority, data):")
        for priority, data in sorted(self.heap):
            print(f"  ({priority}, {data})")
# Example usage
if __name__ == "__main__":
    pq = PriorityQueue()
    # Enqueue elements with priorities
    pq.enqueue("Task A", 3)
    pq.enqueue("Task B", 1)
    pq.enqueue("Task C", 2)
    pq.enqueue("Task D", 1)
    print("Initial Priority Queue:")
    pq.display()
    print("\nDequeuing elements:")
    while pq.heap:
        priority, data = pq.dequeue()
        print(f"Dequeued: ({priority}, {data})")
```

Output:

```
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\p
Initial Priority Queue:
Priority Queue (priority, data):
  (1, Task B)
  (1, Task D)
  (2, Task C)
  (3, Task A)

Dequeuing elements:
Dequeued: (1, Task B)
Dequeued: (1, Task D)
Dequeued: (2, Task C)
Dequeued: (3, Task A)
PS D:\Quick Heal\Microsoft VS Code>
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using

collections.deque.

Sample Input Code:

class DequeDS:

pass

Expected Output:

- **Insert and remove from both ends with docstrings.**

Code:

Users > shyam > ai assistes code > ass13.py > ...

```python
#generate code a DequeDS that implements a double-ended queue (deque) using collections.de
from collections import deque
class DequeDS:
    """
    A double-ended queue (Deque) implementation using Python's collections.deque.

    Supports insertion and removal from both ends.
    """
    def __init__(self):
        """Initialize an empty deque."""
        self.deque = deque()
    def insert_front(self, item):
        """
        Insert an item at the front of the deque.

        Parameters
        ----------
        item : any
            The element to add to the front.
        """
        self.deque.appendleft(item)
        print(f"Inserted {item} at front → Deque: {list(self.deque)}")
    def insert_rear(self, item):
        """
        Insert an item at the rear (end) of the deque.

        Parameters
        ----------
        item : any
            The element to add to the rear.
        """
        self.deque.append(item)
        print(f"Inserted {item} at rear → Deque: {list(self.deque)}")
    def remove_front(self):
        """
        Remove and return the item from the front of the deque.

        Returns
        -------
        item : any
            The removed element, or None if deque is empty.
        """
        if self.is_empty():
            print("Deque is empty! Cannot remove from front.")
            return None
```

```python
    def remove_front(self):
        print(f"Removed {item} from front → Deque: {list(self.deque)}")
        return item
    def remove_rear(self):
        """
        Remove and return the item from the rear of the deque.

        Returns
        -------
        item : any
            The removed element, or None if deque is empty.
        """
        if self.is_empty():
            print("Deque is empty! Cannot remove from rear.")
            return None
        item = self.deque.pop()
        print(f"Removed {item} from rear → Deque: {list(self.deque)}")
        return item
    def is_empty(self):
        """
        Check if the deque is empty.

        Returns
        -------
        bool
            True if deque is empty, False otherwise.
        """
        return len(self.deque) == 0
    def display(self):
        """Display current elements in the deque."""
        print("Current Deque:", list(self.deque))
# Demonstration
if __name__ == "__main__":
    dq = DequeDS()
    # Insert elements
    dq.insert_rear(10)
    dq.insert_front(20)
    dq.insert_rear(30)
    dq.insert_front(40)
    # Remove elements
    dq.remove_front()
    dq.remove_rear()
    # Display final state
```

Output:

```
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\python.ex
Inserted 10 at rear → Deque: [10]
Inserted 20 at front → Deque: [20, 10]
Inserted 30 at rear → Deque: [20, 10, 30]
Inserted 40 at front → Deque: [40, 20, 10, 30]
Removed 40 from front → Deque: [20, 10, 30]
Removed 30 from rear → Deque: [20, 10]
Current Deque: [20, 10]
PS D:\Quick Heal\Microsoft VS Code>
```

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.

2. Event Registration System – Manage participants in events with quick search and removal.

3. Library Book Borrowing – Keep track of available books and their due dates.

4. Bus Scheduling System – Maintain bus routes and stop connections.

5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- **For each feature, select the most appropriate data structure from the list below:**
  - **Stack**
  - **Queue**
  - **Priority Queue**
  - **Linked List**
  - **Binary Search Tree (BST)**
  - **Graph**
  - **Hash Table**
  - **Deque**
- **Justify your choice in 2–3 sentences per feature.**
- **Implement one selected feature as a working Python program with AI-assisted code generation.**

Expected Output:

- **A table mapping feature → chosen data structure → justification.**
- **A functional Python program implementing the chosen feature**

with comments and docstrings.

```python
'''
ceate a Python program for a Cafeteria Order Queue.
Use a queue (collections.deque) to serve students in the order they arrive (FIFO).
Include methods to:
Add a student to the queue.
Serve a student from the front.
Check if the queue is empty.
Display the current queue.
Add docstrings for each method and a small demonstration.
'''
from collections import deque
class CafeteriaQueue:
    """
    A class to simulate a cafeteria order queue using FIFO principle.

    Students are served in the order they arrive.
    """
    def __init__(self):
        """Initialize an empty queue for student orders."""
        self.queue = deque()
    def add_student(self, student_name):
        """
        Add a student to the queue.
        Parameters
        ----------
        student_name : str
            Name of the student joining the queue.
        """
        self.queue.append(student_name)
        print(f"{student_name} joined the queue. Current queue: {list(self.queue)}"
    def serve_student(self):
        """
        Serve the student at the front of the queue.

        Returns
        -------
        student_name : str
            Name of the student served, or None if queue is empty.
        """
        if self.is_empty():
            print("No students in the queue to serve.")
            return None
        student_name = self.queue.popleft()
        print(f"Served {student_name}. Current queue: {list(self.queue)}")
        return student_name
```

```python
        print(f"{student_name} joined the queue. Current queue: {list(self.que
    def serve_student(self):
        """
        Serve the student at the front of the queue.

        Returns
        -------
        student_name : str
            Name of the student served, or None if queue is empty.
        """
        if self.is_empty():
            print("No students in the queue to serve.")
            return None
        student_name = self.queue.popleft()
        print(f"Served {student_name}. Current queue: {list(self.queue)}")
        return student_name
    def is_empty(self):
        """
        Check if the cafeteria queue is empty.
        Returns
        -------
        bool
            True if the queue is empty, False otherwise.
        """
        return len(self.queue) == 0
    def display_queue(self):
        """Display the current students in the queue."""
        print("Current queue:", list(self.queue))
# Demonstration
if __name__ == "__main__":
    cafeteria = CafeteriaQueue()
    # Students joining the queue
    cafeteria.add_student("Alice")
    cafeteria.add_student("Bob")
    cafeteria.add_student("Charlie")
    # Serve students
    cafeteria.serve_student()
    cafeteria.serve_student()
    # Display remaining queue
    cafeteria.display_queue()
```

Output:

```
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\pyt
Alice joined the queue. Current queue: ['Alice']
Bob joined the queue. Current queue: ['Alice', 'Bob']
Charlie joined the queue. Current queue: ['Alice', 'Bob', 'Charlie']
Served Alice. Current queue: ['Bob', 'Charlie']
Served Bob. Current queue: ['Charlie']
Current queue: ['Charlie']
PS D:\Quick Heal\Microsoft VS Code>
```

Task Description #10: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.

2. Emergency Case Handling – Critical patients must be treated first.

3. Medical Records Storage – Fast retrieval of patient details using ID.

4. Doctor Appointment Scheduling – Appointments sorted by time.

5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- **For each feature, select the most appropriate data structure from the list below:**
  - **Stack**
  - **Queue**
  - **Priority Queue**
  - **Linked List**
  - **Binary Search Tree (BST)**
  - **Graph**
  - **Hash Table**
  - **Deque**
- **Justify your choice in 2–3 sentences per feature.**
- **Implement one selected feature as a working Python program with AI-assisted code generation.**

Expected Output:

- **A table mapping feature → chosen data structure → justification.**
- **A functional Python program implementing the chosen feature with comments and docstrings.**

Code:

```python
class EmergencyQueue:
    """
    A priority queue to handle emergency patients.

    Patients with higher priority (lower numerical value) are treated first.
    """
    def __init__(self):
        """Initialize an empty emergency queue."""
        self.heap = []

    def add_patient(self, patient_name, priority):
        """
        Add a patient with a given priority to the queue.

        Parameters
        ----------
        patient_name : str
            Name of the patient.
        priority : int
            Priority level (lower number = higher priority).
        """
        heapq.heappush(self.heap, (priority, patient_name))
        print(f"Added patient {patient_name} with priority {priority}. Queue: {self.heap}")
    def treat_patient(self):
        """
        Treat the patient with the highest priority (lowest number).

        Returns
        -------
        patient_name : str
            Name of the patient treated, or None if the queue is empty.
        """
        if self.is_empty():
            print("No patients in the emergency queue.")
            return None
        priority, patient_name = heapq.heappop(self.heap)
        print(f"Treated patient {patient_name} with priority {priority}. Queue: {self.heap}")
        return patient_name
    def is_empty(self):
        """
        Check if the emergency queue is empty.
        Returns
        -------
        bool
            True if the queue is empty, False otherwise.
```

```python
    def treat_patient(self):
        Returns
        -------
        patient_name : str
            Name of the patient treated, or None if the queue is empty.
        """
        if self.is_empty():
            print("No patients in the emergency queue.")
            return None
        priority, patient_name = heapq.heappop(self.heap)
        print(f"Treated patient {patient_name} with priority {priority}. Queue: {self.heap}")
        return patient_name
    def is_empty(self):
        """
        Check if the emergency queue is empty.
        Returns
        -------
        bool
            True if the queue is empty, False otherwise.
        """
        return len(self.heap) == 0
    def display_queue(self):
        """Display the current patients in the emergency queue."""
        print("Current emergency queue:", self.heap)
# Demonstration
if __name__ == "__main__":
    emergency_queue = EmergencyQueue()
    # Add patients with varying priorities
    emergency_queue.add_patient("Alice", 2)
    emergency_queue.add_patient("Bob", 1)       # Highest priority
    emergency_queue.add_patient("Charlie", 3)
    # Treat patients
    emergency_queue.treat_patient()
    emergency_queue.treat_patient()
    # Display remaining queue
    emergency_queue.display_queue()
```

Output:

```
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\python.exe "c
Added patient Alice with priority 2. Queue: [(2, 'Alice')]
Added patient Bob with priority 1. Queue: [(1, 'Bob'), (2, 'Alice')]
Added patient Charlie with priority 3. Queue: [(1, 'Bob'), (2, 'Alice'), (3, 'Charlie')]
Treated patient Bob with priority 1. Queue: [(2, 'Alice'), (3, 'Charlie')]
Treated patient Alice with priority 2. Queue: [(3, 'Charlie')]
Current emergency queue: [(3, 'Charlie')]
PS D:\Quick Heal\Microsoft VS Code>
```

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings

code:

```python
'''
Create a Python program for a Smart Traffic Management System feature: Emergency Vehicle Priority Handling.
Use a Priority Queue (heapq) so that emergency vehicles with higher priority (like ambulances) move first.
'''
import heapq
class EmergencyVehicleQueue:
    """
    A priority queue to manage emergency vehicles in traffic.

    Vehicles with higher priority (lower numerical value) are given right of way.
    """

    def __init__(self):
        """Initialize an empty priority queue for emergency vehicles."""
        self.heap = []

    def add_vehicle(self, vehicle_name, priority):
        """
        Add an emergency vehicle to the queue.

        Parameters
        ----------
        vehicle_name : str
            Name or type of the emergency vehicle.
        priority : int
            Priority level (lower number = higher priority).
        """
        heapq.heappush(self.heap, (priority, vehicle_name))
        print(f"Added vehicle {vehicle_name} with priority {priority}. Queue: {self.heap}")

    def allow_passage(self):
        """
        Allow the highest-priority vehicle to move first.

        Returns
        -------
        vehicle_name : str
            Name of the vehicle allowed to pass, or None if queue is empty.
        """
```

```python
    def allow_passage(self):
        if self.is_empty():
            print("No emergency vehicles in the queue.")
            return None
        priority, vehicle_name = heapq.heappop(self.heap)
        print(f"Vehicle {vehicle_name} allowed to pass (priority {priority}). Queue: {self.heap}")
        return vehicle_name

    def is_empty(self):
        """
        Check if the emergency vehicle queue is empty.

        Returns
        -------
        bool
            True if the queue is empty, False otherwise.
        """
        return len(self.heap) == 0

    def display_queue(self):
        """Display all vehicles currently in the queue."""
        print("Current emergency vehicle queue:", self.heap)


# Demonstration
if __name__ == "__main__":
    traffic_queue = EmergencyVehicleQueue()

    # Add emergency vehicles with different priorities
    traffic_queue.add_vehicle("Ambulance A", 1)
    traffic_queue.add_vehicle("Fire Truck F", 2)
    traffic_queue.add_vehicle("Police Car P", 3)

    # Allow vehicles to pass based on priority
    traffic_queue.allow_passage()
    traffic_queue.allow_passage()

    # Display remaining queue
    traffic_queue.display_queue()
```

output:

```
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/shya
Added vehicle Ambulance A with priority 1. Queue: [(1, 'Ambulance A')]
Added vehicle Fire Truck F with priority 2. Queue: [(1, 'Ambulance A'), (2, 'Fire Truck F')]
Added vehicle Police Car P with priority 3. Queue: [(1, 'Ambulance A'), (2, 'Fire Truck F'), (3, 'Police Car P')]
Vehicle Ambulance A allowed to pass (priority 1). Queue: [(2, 'Fire Truck F'), (3, 'Police Car P')]
Vehicle Fire Truck F allowed to pass (priority 2). Queue: [(3, 'Police Car P')]
Current emergency vehicle queue: [(3, 'Police Car P')]
PS D:\Quick Heal\Microsoft VS Code>
```

ask Description #12: Smart E-Commerce Platform – Data Structure

Challenge

An e-commerce company wants to build a Smart Online Shopping System

with:

1. Shopping Cart Management – Add and remove products

dynamically.

2. Order Processing System – Orders processed in the order they are placed.

3. Top-Selling Products Tracker – Products ranked by sales count.

4. Product Search Engine – Fast lookup of products using product ID.

5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- **For each feature, select the most appropriate data structure from the list below:**
  - **Stack**
  - **Queue**
  - **Priority Queue**
  - **Linked List**
  - **Binary Search Tree (BST)**
  - **Graph**
  - **Hash Table**
  - **Deque**
- **Justify your choice in 2–3 sentences per feature.**
- **Implement one selected feature as a working Python program with AI-assisted code generation.**

Expected Output:

- **A table mapping feature → chosen data structure → justification.**
- **A functional Python program implementing the chosen feature with comments and docstrings.**

Code:

```python
'''
create a Python program for an E-Commerce Order Processing System using a Queue (collections.deque).
Add methods to add an order, process the next order, check if the queue is empty, and display current orders
'''
from collections import deque
class OrderQueue:
    """
    A queue system for processing customer orders
    in the order they are received (FIFO).
    """
    def __init__(self):
        """Initialize an empty order queue."""
        self.orders = deque()

    def add_order(self, order_id):
        """
        Add a new order to the queue.

        Parameters:
        order_id (str): The ID of the order.
        """
        self.orders.append(order_id)
        print(f"Order {order_id} added to the queue.")
    def process_order(self):
        """
        Process the next order in the queue.
        """
        if self.is_empty():
            print("No orders to process.")
        else:
            order = self.orders.popleft()
            print(f"Processing order {order}")
    def is_empty(self):
        """
        Check if the order queue is empty.

        Returns:
        bool: True if empty, otherwise False.
        """
        return len(self.orders) == 0
    def display_orders(self):
        """Display all current orders in the queue."""
        print("Current Orders:", list(self.orders))
# Demonstration
```

```python
        print("Current Orders:", list(self.orders))
# Demonstration
if __name__ == "__main__":
    shop = OrderQueue()

    shop.add_order("ORD101")
    shop.add_order("ORD102")
    shop.add_order("ORD103")
    shop.display_orders()
    shop.process_order()
    shop.process_order()
    shop.display_orders()
```

Output:

```
Current emergency vehicle queue: [(3, 'Police Car P')]
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\p
Order ORD101 added to the queue.
Order ORD102 added to the queue.
Order ORD103 added to the queue.
Current Orders: ['ORD101', 'ORD102', 'ORD103']
Processing order ORD101
Processing order ORD102
Current Orders: ['ORD103']
PS D:\Quick Heal\Microsoft VS Code>
```

Task #13: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing

2. Urgent Shipment Priority

3. Product Lookup by Code

4. Warehouse Network Connectivity

5. Inventory Add/Remove Operations

Student Task

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• **A functional Python program implementing the chosen feature with comments and docstrings.**

Code:

```python
'''
Create a Python program for a Shipment Processing System using a Queue (collections.deque).
Include methods to add a shipment, process the next shipment, check if the queue is empty, and display all shipments.
Add simple comments, docstrings, and a small demo
'''
from collections import deque
class ShipmentQueue:
    """
    A queue system to process shipments in the order they arrive.
    """
    def __init__(self):
        """Initialize an empty shipment queue."""
        self.queue = deque()
    def add_shipment(self, shipment_id):
        """
        Add a shipment to the queue.
        Parameters:
        shipment_id (str): ID of the shipment.
        """
        self.queue.append(shipment_id)
        print(f"Shipment {shipment_id} added.")
    def process_shipment(self):
        """
        Process the next shipment in the queue.
        """
        if self.is_empty():
            print("No shipments to process.")
        else:
            shipment = self.queue.popleft()
            print(f"Processing shipment {shipment}")
    def is_empty(self):
        """
        Check if the shipment queue is empty.
        Returns:
        bool: True if empty, otherwise False.
        """
        return len(self.queue) == 0
    def display_shipments(self):
        """Display all shipments in the queue."""
        print("Current Shipments:", list(self.queue))
# Demonstration
if __name__ == "__main__":
    logistics = ShipmentQueue()
    logistics.add_shipment("SHP101")
    logistics.add_shipment("SHP102")
    logistics.add_shipment("SHP103")
    logistics.display_shipments()
    logistics.process_shipment()
    logistics.process_shipment()
    logistics.display_shipments()
```

Output:

```
Current Shipments: ['SHP103']
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/shyam/ai assistes code
Shipment SHP101 added.
Shipment SHP102 added.
Shipment SHP103 added.
Current Shipments: ['SHP101', 'SHP102', 'SHP103']
Processing shipment SHP101
Processing shipment SHP102
Current Shipments: ['SHP103']
PS D:\Quick Heal\Microsoft VS Code>
```

ask Description #14: Smart Banking Management System – Data

Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.

2. Loan Approval Priority Handling – High-value or urgent loans

processed first.

3. Account Information Lookup – Fast retrieval of customer details using Account Number.

4. Transaction History Management – Maintain chronological

transaction records.

5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program

with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```python
Create a Python program for a Shipment Processing System using a Queue (collections.deque).
Include methods to add a shipment, process the next shipment, check if the queue is empty, and display all shipments.
Add simple comments, docstrings, and a small demo
'''
from collections import deque
class BankQueue:
    """
    A queue system to manage customer service tokens in a bank.
    Customers are served in the order they arrive (FIFO).
    """
    def __init__(self):
        """Initialize an empty queue."""
        self.queue = deque()
    def add_customer(self, name):
        """
        Add a customer to the service queue.

        Parameters:
        name (str): Customer name.
        """
        self.queue.append(name)
        print(f"{name} joined the queue.")
    def serve_customer(self):
        """
        Serve the next customer in the queue.
        """
        if self.is_empty():
            print("No customers in the queue.")
        else:
            customer = self.queue.popleft()
            print(f"Serving customer: {customer}")
    def is_empty(self):
        """
        Check if the queue is empty.

        Returns:
        bool: True if empty, otherwise False.
        """
        return len(self.queue) == 0
    def display_queue(self):
        """Display current customers in the queue."""
        print("Current Queue:", list(self.queue))
# Demonstration
if __name__ == "__main__":
    bank = BankQueue()

    bank.add_customer("Alice")
    bank.add_customer("Bob")
    bank.add_customer("Charlie")

    bank.display_queue()

    bank.serve_customer()
    bank.serve_customer()
```

Output:

```
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\pytho
Alice joined the queue.
Bob joined the queue.
Charlie joined the queue.
Current Queue: ['Alice', 'Bob', 'Charlie']
Serving customer: Alice
Serving customer: Bob
Current Queue: ['Charlie']
PS D:\Quick Heal\Microsoft VS Code>
```

Task Description #15: Online Learning Management System – Data Structure Selection

An online education platform wants to develop a Learning Management System that handles:

1. Student Login Authentication – Quick validation of user credentials.

2. Assignment Submission Queue – Submissions processed in order of upload.

3. Top Performer Ranking – Rank students based on scores.

4. Course Prerequisite Structure – Represent subject dependencies.

5. Undo Feature in Online Editor – Reverse recent actions. Student Task:

- **Select the most appropriate data structure for each feature from the given list.**
- **Justify your selection in 2–3 sentences per feature.**
- **Implement one selected feature in Python using AI-assisted code generation.**

Expected Output:

- **Feature → Data Structure → Justification table.**
- **Functional Python program with proper documentation.**

Code:

```python
Create a Python program for an Online Editor with Undo Feature using a Stack.
Include methods to add text, undo the last action, check if the stack is empty, and display current content.
Add simple comments, docstrings, and a small demo
'''
class UndoEditor:
    """
    A simple text editor that supports undo functionality
    using a stack.
    """
    def __init__(self):
        """Initialize the editor with an empty stack."""
        self.stack = []
    def write(self, text):
        """
        Add text to the editor.
        Parameters:
        text (str): The text to add.
        """
        self.stack.append(text)
        print(f"Added: {text}")
    def undo(self):
        """
        Undo the most recent action.
        """
        if self.is_empty():
            print("Nothing to undo.")
        else:
            removed = self.stack.pop()
            print(f"Undo action: Removed '{removed}'")
    def is_empty(self):
        """
        Check if there are actions to undo.
        Returns:
        bool: True if empty, otherwise False.
        """
        return len(self.stack) == 0
    def display(self):
        """Display current content."""
        print("Current Content:", " ".join(self.stack))
# Demonstration
if __name__ == "__main__":
    editor = UndoEditor()
    editor.write("Hello")
    editor.write("World")
    editor.display()
```

Output:

```
> & C:\Users\shyam\AppDat
Added: Hello
Added: World
Current Content: Hello World
Undo action: Removed 'World'
Current Content: Hello
PS D:\Quick Heal\Microsoft VS Code>
```

Task Description #16: Smart Airport Management System – Data Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.

2. Emergency Landing Priority – Aircraft in emergency get priority.

3. Flight Information Lookup – Fast search using Flight ID.

4. Air Route Connectivity – Airports connected via routes.

5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

- **Choose suitable data structures from the provided list.**
- **Provide 2–3 sentence justification per feature.**
- **Implement one feature in Python with comments and docstrings using AI assistance.**

Expected Output:

- **Mapping table.**
- **Working Python implementation.**

Code:

```
'''
Create a Python program for an Airport Passenger Check-In System using a Queue (collections.deque).
Include methods to add a passenger, check in the next passenger, check if the queue is empty, and display the queue.
Add simple comments, docstrings, and a small demo
'''
from collections import deque
class PassengerQueue:
    """
    A queue system for managing passenger check-in at the airport.
    Passengers are processed in the order they arrive (FIFO).
    """
    def __init__(self):
        """Initialize an empty passenger queue."""
        self.queue = deque()
    def add_passenger(self, name):
        """
        Add a passenger to the check-in queue.
        Parameters:
        name (str): Passenger name.
        """
        self.queue.append(name)
        print(f"Passenger {name} added to the queue.")
    def check_in(self):
        """
        Process the next passenger in the queue.
        """
        if self.is_empty():
            print("No passengers in the queue.")
        else:
            passenger = self.queue.popleft()
            print(f"Checking in passenger: {passenger}")
    def is_empty(self):
        """
        Check if the queue is empty.
        Returns:
        bool: True if empty, otherwise False.
        """
        return len(self.queue) == 0
    def display_queue(self):
        """Display all passengers waiting in the queue."""
        print("Current Passenger Queue:", list(self.queue))
# Demonstration
if __name__ == "__main__":
    airport = PassengerQueue()
    airport.add_passenger("Alice")
```

```python
# Demonstration
if __name__ == "__main__":
    airport = PassengerQueue()
    airport.add_passenger("Alice")
    airport.add_passenger("Bob")
    airport.add_passenger("Charlie")
    airport.display_queue()
    airport.check_in()
    airport.check_in()
    airport.display_queue()
```

Output:

```
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\python.exe
Passenger Alice added to the queue.
Passenger Bob added to the queue.
Passenger Charlie added to the queue.
Current Passenger Queue: ['Alice', 'Bob', 'Charlie']
Checking in passenger: Alice
Checking in passenger: Bob
Current Passenger Queue: ['Alice', 'Bob', 'Charlie']
Checking in passenger: Alice
Checking in passenger: Bob
Checking in passenger: Bob
Current Passenger Queue: ['Charlie']
PS D:\Quick Heal\Microsoft VS Code>
```

Task Description #17: Smart Social Media Platform – Data Structure Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.

2. Trending Hashtags Tracker – Rank hashtags by usage frequency.

3. User Profile Search – Quick lookup using username.

4. Friend Network Representation – Represent user connections.

5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

- **Select appropriate data structures from the list.**
- **Justify your choices clearly.**
- **Implement one selected feature as a Python program using AI- assisted code generation.**

Expected Output:

- **Table mapping features to data structures with justification.**

• **Working Python code with docstrings and comments.**

Code:

```python
Create a Python program for a Social Media Message System with Undo Feature using a Stack.
Include methods to send a message, undo the last message, check if the stack is empty, and display all messages.
Add simple comments, docstrings, and a small demo
'''
class MessageStack:
    """
    A simple message system that supports undo functionality
    using a stack data structure.
    """
    def __init__(self):
        """Initialize an empty stack for messages."""
        self.stack = []
    def send_message(self, message):
        """
        Send a message and store it in the stack.
        Parameters:
        message (str): The message content.
        """
        self.stack.append(message)
        print(f"Message sent: {message}")
    def undo_message(self):
        """
        Undo the last sent message.
        """
        if self.is_empty():
            print("No messages to undo.")
        else:
            removed = self.stack.pop()
            print(f"Message undone: {removed}")
    def is_empty(self):
        """
        Check if there are messages in the stack.
        Returns:
        bool: True if empty, otherwise False.
        """
        return len(self.stack) == 0
    def display_messages(self):
        """Display current messages."""
        print("Messages:", self.stack)
# Demonstration
if __name__ == "__main__":
    chat = MessageStack()
    chat.send_message("Hello")
    chat.send_message("How are you?")
```

Output:

```
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\python.exe  c:/Users/shyam/ai assistes code/a
Message sent: Hello
Message sent: How are you?
Messages: ['Hello', 'How are you?']
Message undone: How are you?
Messages: ['Hello']
Message undone: How are you?
Messages: ['Hello']
PS D:\Quick Heal\Microsoft VS Code>
```

Task #18: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment

2. Medical Emergency Handling

3. Student Record Retrieval

4. Hostel Block Connectivity

5. Visitor Entry Log

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

Output:

```python
'''
Create a Python program for a Social Media Message System with Undo Feature using a Stack.
Include methods to send a message, undo the last message, check if the stack is empty, and display all messages.
Add simple comments, docstrings, and a small demo
'''
from collections import deque

class RoomAllotmentQueue:
    """
    A queue system to manage hostel room allotment requests.
    Students are allotted rooms in the order they apply.
    """
    def __init__(self):
        """Initialize an empty queue for room requests."""
        self.queue = deque()
    def add_request(self, student_name):
        """
        Add a student request for hostel room.
        Parameters:
        student_name (str): Name of the student.
        """
        self.queue.append(student_name)
        print(f"{student_name} added to room allotment queue.")
    def allot_room(self):
        """
        Allot a room to the next student in the queue.
        """
        if self.is_empty():
            print("No room requests available.")
        else:
            student = self.queue.popleft()
            print(f"Room allotted to {student}")
    def is_empty(self):
        """
        Check if the queue is empty.
        Returns:
        bool: True if queue is empty.
        """
        return len(self.queue) == 0
    def display_queue(self):
        """Display current students waiting for rooms."""
        print("Room Request Queue:", list(self.queue))
# Demonstration
if __name__ == "__main__":
    hostel = RoomAllotmentQueue()
```

Output:

```
Messages: ['Hello']
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/shyam/ai
Rahul added to room allotment queue.
Anita added to room allotment queue.
Kiran added to room allotment queue.
Room Request Queue: ['Rahul', 'Anita', 'Kiran']
Room allotted to Rahul
Room allotted to Anita
Room Request Queue: ['Kiran']
PS D:\Quick Heal\Microsoft VS Code>
```

Task #19: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling

2. Fault Priority Handling

3. Consumer Record Lookup

4. Power Grid Connectivity

5. Meter Reading History Tracking

Student Task

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Code:

```python
'''
Create a Python program for an Electricity Complaint Handling System using a Queue (collections.deque).
Include methods to add a complaint, resolve the next complaint, check if the queue is empty, and display all complaints.
Add simple comments, docstrings, and a small demo
'''
from collections import deque
class ComplaintQueue:
    """
    A queue system to manage electricity consumer complaints.
    Complaints are handled in the order they are received (FIFO).
    """
    def __init__(self):
        """Initialize an empty complaint queue."""
        self.queue = deque()
    def add_complaint(self, complaint):
        """
        Add a new complaint to the queue.
        Parameters:
        complaint (str): Description or ID of the complaint.
        """
        self.queue.append(complaint)
        print(f"Complaint added: {complaint}")
    def resolve_complaint(self):
        """
        Resolve the next complaint in the queue.
        """
        if self.is_empty():
            print("No complaints to resolve.")
        else:
            complaint = self.queue.popleft()
            print(f"Resolving complaint: {complaint}")
    def is_empty(self):
        """
        Check if the complaint queue is empty.
        Returns:
        bool: True if empty, otherwise False.
        """
        return len(self.queue) == 0
    def display_complaints(self):
        """Display all pending complaints."""
        print("Pending Complaints:", list(self.queue))
# Demonstration
```

Output:

```
PS D:\Quick Heal\Microsoft VS Code> & C:\Users\shyam\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/shyam/ai assistes co
Complaint added: Power outage in Block A
Complaint added: Voltage fluctuation in Street 5
Complaint added: Meter not working in House 12
Pending Complaints: ['Power outage in Block A', 'Voltage fluctuation in Street 5', 'Meter not working in House 12']
Resolving complaint: Power outage in Block A
Resolving complaint: Voltage fluctuation in Street 5
Pending Complaints: ['Meter not working in House 12']
PS D:\Quick Heal\Microsoft VS Code>
```